جلسهٔ ۱۴ — تشخیص و مکان‌یابیِ اشیا (Object Detection & Localization)

جزوهٔ درس — هفتهٔ چهاردهم

فهرست مطالب

۱ سه وظیفه: طبقه‌بندی، مکان‌یابی، تشخیص ۳

۲ جعبهٔ محصورکننده فقط چهار عدد است ۳

۳ مکان‌یابی به‌مثابهٔ رگرسیون ۴

۴ ایدهٔ پنجرهٔ لغزان ۵

۱.۴ هرمِ تصویر برای مقیاس‌های مختلف . . . ۵

۲.۴ هزینهٔ محاسباتی . . . ۵

۵ IoU: اندازه‌گیریِ هم‌پوشانی ۶

۱.۵ یک مثالِ عددیِ کامل . . . ۶

۲.۵ خواندنِ مقادیرِ IoU . . . ۷

۶ NMS: نگه‌داشتنِ بهترین جعبه ۷

۱.۶ یک مثالِ عددیِ NMS . . . ۷

۷ از پنجرهٔ لغزان تا آشکارسازهای مدرن ۸

۱.۷ گامِ اول: پیشنهاد بده، جارو نکن . . . ۸

۲.۷ گامِ دوم: تک‌شات . . . ۸

۸ YOLO چطور تصویر را می‌بیند ۸

۱.۸ ایدهٔ شبکه (grid) ۸
۲.۸ هر سلول چه پیش‌بینی می‌کند ۹
۳.۸ تانسورِ خروجی، شمرده ۹
۴.۸ جعبه‌های لنگر (anchor boxes) ۹
۵.۸ آموزش و استنتاج ۹

۹ ارزیابی: از precision/recall تا mAP ۱۰

۱.۹ یک مثالِ عددیِ precision/recall ۱۰
۲.۹ AP و mAP ۱۰

۱۰ کدِ IoU و NMS از صفر ۱۱

۱۱ پیوند با DocScan ۱۲

۱۲ دومرحله‌ای در برابرِ تک‌شات — مقایسهٔ صادقانه ۱۲

۱۳ تمرینِ کلاسی و پاسخ ۱۳

۱۴ فهرستِ عیب‌یابیِ عملیِ تشخیص ۱۳

۱۵ جمع‌بندی ۱۴

نگاهِ کلی به این جلسه

در هفتهٔ ۱۳ یک CNN ساختیم که به پرسشِ «چه چیزی؟» پاسخ می‌دهد: این تصویر کدام رقم است. اما در دنیای واقعی، عکسِ کنتور فقط یک رقمِ برش‌خوردهٔ × نیست؛ یک قابِ بزرگ است که کنتور جایی در آن قرار دارد. این هفته به پرسشِ دوم پاسخ می‌دهیم: «کجا؟» — یعنی دورِ هر شیء یک **جعبهٔ محصورکننده** (bounding box) می‌کشیم. ترکیبِ «چه چیزی + کجا» همان **تشخیصِ شیء** (object detection) است، اسبِ‌کاریِ بینایی ماشینِ مدرن.

یادداشت

پیوند با DocScan: پیش از آنکه خوانندهٔ ارقامِ هفتهٔ ۱۳ بتواند کاری بکند، باید بدانیم رقم‌ها کجای عکس‌اند. زنجیرهٔ کاملِ DocScan این است: نخست **سندِ کنتور** را پیدا کن، سپس **نوارِ ارقام** را مکان‌یابی کن، و در پایان آن برش را به خوانندهٔ هفتهٔ ۱۳ بده. تشخیص، همان حلقهٔ گم‌شدهٔ «پیدایش کن» است.

یک تشبیهِ ساده

تصور کنید یک بازرسِ پستِ شلوغ روی میزی پر از بسته ایستاده است. طبقه‌بندی مثلِ این است که بپرسیم «آیا روی این میز بسته‌ای هست؟ بله/خیر». مکان‌یابی یعنی «یک بسته هست، دقیقاً اینجا» و دور یک بسته کادر بکشیم. اما تشخیص کارِ واقعیِ بازرس است: «شش بسته اینجاست، هرکدام در این مختصات، و این یکی شکستنی است». تفاوتِ اساسی این است که بازرس از پیش نمی‌داند چند بسته روی میز است؛ باید همه را پیدا کند، دورشان کادر بکشد و کادرهای تکراری را حذف کند. همین «تعدادِ نامعلومِ خروجی‌ها» چیزی است که تشخیص را از طبقه‌بندی سخت‌تر می‌کند.

۱ سه وظیفه: طبقه‌بندی، مکان‌یابی، تشخیص

بیایید سه وظیفهٔ نزدیک‌به‌هم را دقیق از هم جدا کنیم:

وظیفه	خروجی	مثالِ کنتور
طبقه‌بندی (classification)	یک برچسب برای کلِ تصویر	«این برش یک ۷ است»
مکان‌یابی (localization)	یک برچسب + یک جعبه	«یک کنتور، اینجا»
تشخیص (detection)	چندین برچسب + جعبه	«کنتور اینجا، عقربه آنجا»

نکتهٔ کلیدی

تعریفِ عملیِ تشخیص: «طبقه‌بندی‌ای که آن را همه‌جای تصویر اجرا کرده‌ایم، و سپس نتیجه را به چند جعبهٔ خوب تمیز کرده‌ایم». اگر این جمله را بفهمید، بقیهٔ هفته فقط جزئیاتِ همین ایده است.

۲ جعبهٔ محصورکننده فقط چهار عدد است

یک جعبه را می‌توان به دو صورت نمایش داد:

- **صورتِ گوشه‌ای:** (x_1, y_1, x_2, y_2) — گوشهٔ بالا-چپ و پایین-راست.

- **صورتِ مرکزی:** (c_x, c_y, w, h) — مرکز، عرض و ارتفاع. آشکارسازها معمولاً همین را پیش‌بینی می‌کنند.

تبدیل بینِ این دو ساده است: $w = x_۲ - x_۱$، $h = y_۲ - y_۱$، $c_x = x_۱ + w/۲$ و $c_y = y_۱ + h/۲$.

ریاضیات

مساحتِ یک جعبه: $\text{area} = (x_۲ - x_۱)(y_۲ - y_۱)$. مثلاً جعبهٔ $(۱, ۱, ۵, ۴)$ مساحتِ $(۵ - ۱)(۴ - ۱) = ۱۲$ دارد.

۳ مکان‌یابی به‌مثابهٔ رگرسیون

برای مکان‌یابیِ یک شیء، همان ستون‌فقراتِ CNN هفتهٔ ۱۳ را می‌گیریم و روی آن **دو سر (head)** می‌گذاریم:

- **سرِ طبقه‌بندی:** امتیازِ کلاس‌ها را می‌دهد (softmax، مثلِ قبل).
- **سرِ رگرسیونِ جعبه:** چهار عدد (c_x, c_y, w, h) را می‌دهد.

تفاوتِ کلیدی: طبقه‌بندی یک دسته پیش‌بینی می‌کند، اما رگرسیون اعداد پیش‌بینی می‌کند. و جعبه چیزی جز چهار عدد نیست، پس مکان‌یابی یک مسئلهٔ رگرسیون است.

ریاضیات

زیانِ جعبه با اعدادِ واقعی. فرض کنید جعبهٔ پیش‌بینی‌شده $\hat{b} = (۳۰, ۲۰, ۱۰۰, ۴۰)$ و جعبهٔ واقعی $b = (۳۲, ۱۸, ۹۶, ۴۴)$ باشد. زیانِ مربعی:

$$(۳۰-۳۲)^۲ + (۲۰-۱۸)^۲ + (۱۰۰-۹۶)^۲ + (۴۰-۴۴)^۲ = ۴ + ۴ + ۱۶ + ۱۶ = ۴۰$$

گرادیان کاهشی این عدد را کوچک می‌کند و جعبه را به محلِ درست می‌کشاند — دقیقاً همان سازوکارِ آموزشِ هفتهٔ ۱۳. در عمل به‌جای زیانِ مربعیِ خالص از ۱L smooth استفاده می‌کنیم: برای خطاهای کوچک مربعی و برای خطاهای بزرگ خطی است تا نسبت به داده‌های پرت مقاوم بماند.

هشدار / دام

یک سرِ رگرسیون فقط یک شیء را مکان‌یابی می‌کند. اما یک عکسِ واقعی ممکن است بدنهٔ کنتور، چند عقربه و یک پلاکِ سریال داشته باشد و ما از پیش نمی‌دانیم چند جعبه باید خروجی بدهیم. این «تعدادِ متغیرِ خروجی‌ها» هستهٔ سختیِ تشخیص است.

۴ ایدهٔ پنجرهٔ لغزان

ساده‌ترین راهِ رسیدن از مکان‌یابیِ تک‌شیء به تشخیصِ چندشیء، **پنجرهٔ لغزان** (sliding window) است:

۱. یک اندازهٔ پنجره انتخاب کن.

۲. آن را روی تصویر بلغزان (مثلِ کرنلِ هفتهٔ ۳، اما بزرگ‌تر).

۳. روی هر برش، طبقه‌بند را اجرا کن: «آیا اینجا نوارِ ارقام هست؟ بله/خیر + امتیاز».

۴. هر پنجره‌ای که امتیازِ بالا گرفت را نگه دار.

۱.۴ هرمِ تصویر برای مقیاس‌های مختلف

یک اندازهٔ پنجره فقط اشیای هم‌اندازهٔ خودش را پیدا می‌کند. راهِ حل: تصویر را چند بار کوچک کن (یک **هرم**) و همان پنجره را روی هر سطح بلغزان. یک شیءِ بزرگ در سطوحِ بالای هرم کوچک به‌نظر می‌رسد و اکنون در پنجره جا می‌شود. این‌گونه جست‌وجویِ ناوردا به مقیاس را رایگان به‌دست می‌آوریم.

۲.۴ هزینهٔ محاسباتی

ریاضیات

تعدادِ تقریبیِ پنجره‌ها:

$$\text{windows} \approx \frac{W \cdot H}{s^2} \times L$$

که $W \times H$ اندازهٔ تصویر، s گامِ حرکت و L تعدادِ سطوحِ هرم است. برای تصویرِ ۶۴۰×۴۸۰ با گامِ $s = ۸$ و $L = ۵$ سطح:

$$\frac{۶۴۰ \cdot ۴۸۰}{۶۴} \times ۵ = ۴۸۰۰ \times ۵ = ۲۴۰۰۰$$

یعنی حدودِ ۲۴٬۰۰۰ فراخوانیِ طبقه‌بند برای هر تصویر — و این فقط با یک نسبتِ‌ابعادِ پنجره است.

یادداشت

پنجرهٔ لغزان کند است، اما مفهوماً همان تشخیص است. هر آشکارسازِ مدرن، نسخهٔ کارآمدترِ همین جاروبِ ساده است. آن را به‌عنوانِ مدلِ ذهنی نگه دارید.

۵ IoU: اندازه‌گیریِ هم‌پوشانی

پنجرهٔ لغزان روی چند پنجرهٔ نزدیک به یک شیء آتش می‌کند؛ شما یک جعبه می‌خواستید اما پنج جعبهٔ هم‌پوشان گرفتید. برای پاک‌سازی، ابتدا باید «میزانِ هم‌پوشانیِ دو جعبه» را اندازه بگیریم. ابزارِ استاندارد، **اشتراک بر اجتماع** (Intersection over Union، IoU) است:

ریاضیات

$$\text{اجتماع} = A_۱ + A_۲ - \text{اشتراک}, \qquad \text{IoU} = \frac{\text{مساحتِ اشتراک}}{\text{مساحتِ اجتماع}}$$

مقدارِ ۰ یعنی هیچ هم‌پوشانی، و مقدارِ ۱ یعنی دو جعبهٔ کاملاً یکسان.

۱.۵ یک مثالِ عددیِ کامل

بیایید IoU را برای دو جعبهٔ مشخص با دست حساب کنیم. جعبه‌ها به‌صورتِ $(x_۱, y_۱, x_۲, y_۲)$: $A = (۱, ۱, ۵, ۴)$ و $B = (۳, ۲, ۷, ۶)$.

ریاضیات

مستطیلِ اشتراک از بیشینهٔ گوشه‌های چپ-بالا تا کمینهٔ گوشه‌های راست-پایین:

$$x: \ [\max(۱, ۳), \min(۵, ۷)] = [۳, ۵] \Rightarrow \text{عرض} = ۲$$

$$y: \ [\max(۱, ۲), \min(۴, ۶)] = [۲, ۴] \Rightarrow \text{ارتفاع} = ۲$$

$$\text{اشتراک} = ۲ \times ۲ = ۴$$

مساحت‌ها و IoU:

$$A_۱ = (۵-۱)(۴-۱) = ۱۲, \quad A_۲ = (۷-۳)(۶-۲) = ۱۶$$

$$\text{اجتماع} = ۱۲ + ۱۶ - ۴ = ۲۴, \qquad \text{IoU} = \frac{۴}{۲۴} \approx ۰٫۱۶۷$$

نکتهٔ کلیدی

نکتهٔ ظریفِ برنامه‌نویسی: عرض و ارتفاعِ اشتراک باید در صفر مهار (clamp) شوند. اگر دو جعبه هم‌پوشانی نداشته باشند، بازه‌ها هم‌دیگر را قطع نمی‌کنند و ممکن است عرض یا ارتفاعِ منفی به‌دست آید؛ باید آن را صفر بگیریم، نه اینکه مساحتِ منفیِ جعلی تولید کنیم. فراموش‌کردنِ همین $\max(۰, ۰)$ رایج‌ترین باگِ پیاده‌سازیِ IoU است.

۲.۵ خواندنِ مقادیرِ IoU

IoU	معنا
۰	جعبه‌ها به‌هم نمی‌رسند
۰٫۱-۰٫۳	هم‌پوشانیِ اندک؛ احتمالاً دو شیءِ متفاوت
۰٫۵	آستانهٔ کلاسیکِ «درست‌بودن» در ارزیابی
۰٫۷-۰٫۹	جعبهٔ دقیق و هم‌ترازِ خوب
۱٫۰	دو جعبهٔ پیکسل-به-پیکسل یکسان

آستانهٔ IoU $\geq$ ۰٫۵ پیش‌فرضِ جامعهٔ بینایی ماشین برای «به‌قدرِ کافی درست» است؛ وظایفِ سخت‌گیرانه‌تر ۰٫۷۵ می‌خواهند.

۶ NMS: نگه‌داشتنِ بهترین جعبه

اکنون که هم‌پوشانی را می‌سنجیم، **سرکوبِ غیربیشینه** (Non-Maximum Suppression، NMS) جعبه‌های تکراری را جمع می‌کند. دستورِ حریصانه، برای هر کلاس:

۱. همهٔ جعبه‌ها را بر اساسِ امتیاز از زیاد به کم مرتب کن.

۲. جعبهٔ سرِ فهرست را بردار و نگه دار.

۳. هر جعبهٔ باقی‌مانده‌ای که اشIoU با آن از آستانهٔ τ (مثلاً ۰٫۵) بیشتر است را حذف کن.

۴. با جعبهٔ بازماندهٔ بعدی تکرار کن تا چیزی نماند.

۱.۶ یک مثالِ عددیِ NMS

سه جعبه با آستانهٔ $\tau = ۰٫۵$:

جعبه	امتیاز	IoU با نگه‌داشته	اقدام
۱B	۰٫۹۰	—	**نگه‌دار** (بالاترین)
۲B	۰٫۸۰	۰٫۷۰ نسبت به ۱B	سرکوب ($> \tau$)
۳B	۰٫۷۵	۰٫۱۰ نسبت به ۱B	**نگه‌دار** (شیءِ دیگر)

از ۱B (بیشترین امتیاز) شروع می‌کنیم. ۲B با آن هم‌پوشانیِ زیاد دارد، پس به‌عنوانِ تکراری حذف می‌شود. ۳B هم‌پوشانیِ ناچیزی دارد، پس یک شیءِ دوم است و آن را نگه می‌داریم. خروجی: {۱B، ۳B}. سه جعبهٔ پرسروصدا به دو جعبهٔ تمیز تبدیل شدند.

هشدار / دام

انتخابِ τ حساس است. τ خیلی کوچک (مثلاً ۰٫۲) دو شیءِ واقعیِ نزدیک‌به‌هم (دو عقربه) را در هم می‌آمیزد و باعثِ حذفِ تشخیص می‌شود؛ τ خیلی بزرگ (مثلاً ۰٫۹) جعبه‌های تکراری را زنده نگه می‌دارد. مقدارِ $\tau \approx$ ۰٫۵ سازشِ رایج است. برای صحنه‌های شلوغ می‌توان از «Soft-NMS» استفاده کرد که به‌جای حذفِ سختِ جعبه، امتیازِ آن را کاهش می‌دهد.

یادداشت

یک اشتباهِ رایج: اجرای NMS بین کلاس‌ها. NMS باید درونِ هر کلاس اجرا شود، وگرنه جعبهٔ پرامتیازِ «کنتور» ممکن است جعبهٔ نزدیکِ «عقربه» را سرکوب کند. همیشه NMS را per-class نگه دارید.

۷ از پنجرهٔ لغزان تا آشکارسازهای مدرن

۱.۷ گامِ اول: پیشنهاد بده، جارو نکن

پنجرهٔ لغزان همهٔ مکان‌ها و مقیاس‌ها را آزمایش می‌کند و بیشترشان پس‌زمینهٔ خالی‌اند. ایدهٔ **پیشنهادِ ناحیه** (region proposal) — مثلاً الگوریتمِ selective search — نخست چند هزار جعبهٔ «شیءمانند» را پیشنهاد می‌دهد و CNN فقط روی همان‌ها اجرا می‌شود. این صرفه‌جوییِ عظیمی نسبت به صدها هزار پنجره است. خانوادهٔ **دومرحله‌ای** (R-CNN → Fast R-CNN → Faster R-CNN) بر همین ایده بنا شده: مرحلهٔ یک «کجا ممکن است شیء باشد؟» و مرحلهٔ دو «در هر پیشنهاد چه چیزی هست + جعبه را دقیق کن».

۲.۷ گامِ دوم: تک‌شات

آشکارسازهای **تک‌شات** (single-shot) مثلِ **YOLO** و **SSD** همه‌چیز را در یک گذرِ روبه‌جلو انجام می‌دهند: بدونِ مرحلهٔ جداگانهٔ پیشنهاد، خودِ CNN مستقیماً جعبه‌ها و کلاس‌ها را خروجی می‌دهد. آن‌قدر سریع که برای ویدئو کافی است — نامِ YOLO یعنی «فقط یک بار نگاه می‌کنی».

۸ YOLO چطور تصویر را می‌بیند

۱.۸ ایدهٔ شبکه (grid)

تصویر را به یک شبکهٔ $S \times S$ تقسیم می‌کنیم. سلولی که مرکزِ یک شیء در آن می‌افتد، مسئولِ تشخیصِ آن شیء است. هر سلول چند جعبه نسبت به خودش و نیز امتیازِ کلاس‌ها را پیش‌بینی می‌کند.

۲.۸ هر سلول چه پیش‌بینی می‌کند

پیش‌بینی	معنا
t_x, t_y	مرکزِ جعبه، به‌صورتِ آفستِ درونِ سلول (بینِ ۰ و ۱)
t_w, t_h	عرض و ارتفاعِ جعبه (نسبت به تصویر یا لنگر)
p_o objectness	«آیا اصلاً شیئی اینجا هست؟»
$p_1..p_C$	«اگر باشد، کدام کلاس است؟»

امتیازِ نهاییِ هر جعبه که به NMS داده می‌شود: $\text{score} = p_o \times p_{\text{class}}$، یعنی حاصل‌ضربِ «شیء بودن» در «احتمالِ کلاس».

۳.۸ تانسورِ خروجی، شمرده

ریاضیات

شکلِ خروجیِ کلِ شبکه:

$$S \times S \times \big(B \cdot ۵ + C\big)$$

که B تعدادِ جعبه‌های هر سلول و عددِ ۵ همان $(t_x, t_y, t_w, t_h, p_o)$ است و C تعدادِ کلاس‌ها. برای مثالِ DocScan با $S = ۷$، $B = ۲$ و $C = ۲$ (سند، نوارِ ارقام):

$$۷ \times ۷ \times (۲ \cdot ۵ + ۲) = ۷ \times ۷ \times ۱۲ = ۵۸۸ \text{ عدد}.$$

همهٔ این ۵۸۸ عدد در یک گذرِ روبه‌جلو تولید می‌شوند — بدونِ هیچ جاروبی.

۴.۸ جعبه‌های لنگر (anchor boxes)

اشیا شکل‌های معمولی دارند: نوارِ ارقام پهن است و یک عقربه تقریباً مربع. **لنگرها** چند شکلِ جعبهٔ ازپیش‌تعیین‌شده در هر سلول‌اند؛ شبکه به‌جای پیش‌بینیِ جعبه از صفر، فقط یک اصلاحِ کوچک روی لنگر پیش‌بینی می‌کند. یادگرفتنِ یک «تلنگر» ساده‌تر از یادگرفتنِ اندازهٔ مطلق است، و لنگرهای مختلف در شکل‌های مختلف تخصص پیدا می‌کنند.

۵.۸ آموزش و استنتاج

هنگامِ آموزش، هر شیءِ واقعی به سلول/لنگری با بهترین IoU نسبت داده می‌شود و زیان از سه بخش تشکیل می‌شود: زیانِ جعبه (رگرسیون)، زیانِ objectness و زیانِ کلاس. چون بیشترِ سلول‌ها پس‌زمینه‌اند، باید زیانِ objectness را با دقت وزن‌دهی کرد وگرنه شبکه یاد می‌گیرد همیشه «هیچ‌چیز» پیش‌بینی کند. هنگامِ استنتاج: گذرِ روبه‌جلو ← تانسور را رمزگشایی کن ← امتیازهای پایین را حذف کن ← جعبه‌های سلول-نسبی را به پیکسلِ مطلق تبدیل کن ← **NMS** per-class بزن. یعنی همان دو ابزارِ جلسهٔ قبل

هنوز پاک‌سازیِ نهایی را انجام می‌دهند.

۹ ارزیابی: از precision/recall تا mAP

یک پیش‌بینی مثبتِ درست (true positive) شمرده می‌شود اگر اشIoU با یک جعبهٔ واقعی $\geq$ آستانه (معمولاً ۰٫۵) باشد و کلاس مطابقت داشته باشد. در غیرِ این‌صورت مثبتِ کاذب (false positive) است و یک شیءِ واقعیِ ازدست‌رفته منفیِ کاذب (false negative). فقط یک پیش‌بینی می‌تواند هر جعبهٔ واقعی را ادعا کند؛ باقی مثبتِ کاذب‌اند.

ریاضیات

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}$$

۱.۹ یک مثالِ عددیِ precision/recall

پنج پیش‌بینی که بر اساسِ امتیاز مرتب شده‌اند، و ۳ شیءِ واقعی. آستانهٔ IoU ۰٫۵ $\geq$ = برخورد:

رتبه	نتیجه	TP تجمعی	precision	recall
۱	TP	۱	$۱/۱ = ۱٫۰۰$	$۱/۳ = ۰٫۳۳$
۲	TP	۲	$۲/۲ = ۱٫۰۰$	$۲/۳ = ۰٫۶۷$
۳	FP	۲	$۲/۳ = ۰٫۶۷$	۰٫۶۷
۴	TP	۳	$۳/۴ = ۰٫۷۵$	$۳/۳ = ۱٫۰۰$
۵	FP	۳	$۳/۵ = ۰٫۶۰$	۱٫۰۰

هرچه جعبه‌های کم‌امتیازتر را می‌پذیریم، recall بالا می‌رود اما precision معمولاً افت می‌کند — همان سازشِ کلاسیک.

۲.۹ AP و mAP

ریاضیات

دقتِ میانگین (Average Precision، AP) مساحتِ زیرِ منحنیِ precision-recall (پس از درون‌یابی) است. **mAP** میانگینِ AP روی همهٔ کلاس‌هاست:

$$\text{mAP} = \frac{1}{C}\sum_{c=1}^{C} \text{AP}_c$$

اصطلاحِ mAP@0.5 یعنی برخوردها به IoU ≥ 0.5 نیاز دارند؛ mAP@[.5:.95] (سبکِ COCO) میانگینِ AP روی چندین آستانهٔ IoU است و سخت‌گیرانه‌تر.

هشدار / دام

دقتِ طبقه‌بندیِ بالا هیچ چیزی دربارهٔ کیفیتِ جعبه نمی‌گوید. تشخیص باید با معیارهای آگاه‌–IoU مثلِ mAP سنجیده شود، نه با دقتِ ساده. جعبه‌ها می‌توانند بد جای‌گذاری شده باشند و باز هم «دقیق» به‌نظر برسند.

۱۰ کدِ IoU و NMS از صفر

```
def iou(a, b):                      # boxes as (x1, y1, x2, y2)
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    iw, ih = max(0, ix2 - ix1), max(0, iy2 - iy1)   # clamp at 0
    inter = iw * ih
    area_a = (a[2]-a[0]) * (a[3]-a[1])
    area_b = (b[2]-b[0]) * (b[3]-b[1])
    return inter / (area_a + area_b - inter + 1e-9)

def nms(boxes, scores, tau(5.0=:
    order = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)
    keep = []
    while order:
        i = order.pop(0)                # take the best
        keep.append(i)
        order = [j for j in order if iou(boxes[i], boxes[j]) < tau]
    return keep
```

مهارِ $\max(0,0)$ کلِ ترفندِ IoU است، و $10^{-9}+$ از تقسیم‌برصفر جلوگیری می‌کند. NMS حریصانه است: بهترین جعبه را نگه می‌دارد، هر چیزِ هم‌پوشانِ بیش از τ را حذف می‌کند و تکرار می‌شود. توجه کنید که این تابع اندیسِ جعبه‌های بازمانده را

برمی‌گرداند.

۱۱ پیوند با DocScan

اکنون زنجیرهٔ کاملِ DocScan را داریم. برای عکسِ کنتور:

۱. پنجرهٔ لغزان (یا یک آشکارسازِ تک‌شات) نوارِ ارقام را مکان‌یابی می‌کند و امتیاز می‌دهد.

۲. NMS خوشهٔ جعبه‌ها را به یک جعبهٔ تمیز فرومی‌کاهد.

۳. همان جعبه دقیقاً برشی است که خوانندهٔ ارقامِ هفتهٔ ۱۳ انتظارش را دارد.

۴. خوانندهٔ CNN رقم‌ها را می‌خواند و مقدارِ کنتور به‌دست می‌آید.

نکتهٔ کلیدی

همان الگو، در مقیاسِ کوچک: جاروب → امتیاز → NMS → یک جعبهٔ تمیز. «پیدایش کن، بعد بخوانش» شعارِ کلِ این هفته است و حلقهٔ گم‌شدهٔ خطِ‌لوله DocScan را کامل می‌کند.

۱۲ دومرحله‌ای در برابرِ تک‌شات — مقایسهٔ صادقانه

	دومرحله‌ای (Faster R-CNN)	تک‌شات (YOLO/SSD)
سرعت	کندتر	بلادرنگ
دقت	معمولاً بالاتر	بسیار نزدیک و روبه‌بهبود
اشیای کوچک	بهتر	تاریخاً ضعیف‌تر
سادگی	زنجیره‌ای از اجزا	یک شبکهٔ واحد

قاعدهٔ سرانگشتی: اگر روی گوشی و بلادرنگ نیاز دارید (مثلِ DocScan)، تک‌شات؛ اگر هر شیءِ ریز را به هر قیمتی می‌خواهید، دومرحله‌ای.

۱۳ تمرینِ کلاسی و پاسخ

تمرین

۱) جعبهٔ $P = (۰, ۰, ۴, ۴)$ و $Q = (۲, ۲, ۶, ۶)$. مساحتِ اشتراک، اجتماع و IoU را حساب کنید.

۲) یک شبکهٔ YOLO با $S = ۷$، $B = ۳$ و $C = ۵$. اندازهٔ تانسورِ خروجی چقدر است؟

۳) چرا باید آشکارساز را با mAP سنجید نه با دقتِ طبقه‌بندی؟

۴) آشکارسازِ شما یک کنتور را سه جعبه گزارش می‌کند. کدام دکمه را می‌چرخانید و چطور؟

پاسخ‌ها: (۱) اشتراک $۴ = ۲ \times ۲ =$؛ مساحت‌ها هرکدام ۱۶؛ اجتماع $۲۸ = ۴ - ۱۶ + ۱۶ =$؛ پس IoU $= ۴/۲۸ \approx ۰٫۱۴$. (۲) $۷ \times ۷ \times (۳ \cdot ۵ + ۵) = ۷ \times ۷ \times ۲۰ = ۹۸۰$ عدد. (۳) دقتِ طبقه‌بندی به محلِ جعبه بی‌اعتناست؛ mAP هم کلاسِ درست و هم جعبهٔ دقیق–IoU را پاداش می‌دهد و سازشِ precision/recall را روی همهٔ آستانه‌ها لحاظ می‌کند. (۴) NMS: آستانهٔ IoU یعنی τ زیادی بالاست؛ آن را پایین بیاورید تا جعبه‌های تکراری سرکوب شوند. (اگر مثبتِ کاذبِ کلی زیاد بود، به‌جایش آستانهٔ اطمینان را بالا می‌بردید — دو دکمهٔ متفاوت برای دو مشکلِ متفاوت.)

۱۴ فهرستِ عیب‌یابیِ عملیِ تشخیص

وقتی آشکارسازِ خودتان را می‌سازید و درست کار نمی‌کند، این فهرست را به‌ترتیب چک کنید:

- **جعبه‌های تکراریِ زیاد؟** آستانهٔ NMS (τ) زیادی بالاست؛ پایینش بیاورید.
- **مثبتِ کاذبِ زیاد؟** آستانهٔ اطمینان را بالا ببرید تا جعبه‌های کم‌امتیاز حذف شوند.
- **دو شیءِ نزدیک در هم ادغام شدند؟** τ زیادی پایین است، یا به Soft-NMS نیاز دارید.
- **IoU همیشه صفر است؟** احتمالاً صورتِ جعبه ($x_۱y_۱x_۲y_۲$) در برابرِ مرکزی) را جابه‌جا گرفته‌اید، یا مهارِ $\max(۰, ۰)$ را فراموش کرده‌اید.
- **شبکه همیشه «هیچ‌چیز» می‌گوید؟** عدمِ توازنِ پس‌زمینه/پیش‌زمینه؛ وزنِ زیانِ objectness را تنظیم کنید.
- **لنگرها به اشیا نمی‌خورند؟** به نوارِ ارقامِ پهن، لنگرِ پهن بدهید.

هشدار / دام

پیش از هر آموزشِ بزرگ، آشکارساز را روی یک تصویرِ واحد امتحان کنید و ببینید آیا می‌تواند همان یک شیء را کامل بیش‌برازش کند. اگر نتواند حتی یک نمونه را حفظ کند، باگی اساسی — معمولاً در نسبت‌دادنِ جعبهٔ واقعی به لنگر یا در تبدیلِ مختصات — دارید.

۱۵ جمع‌بندی

در این جلسه یاد گرفتیم که تشخیص یعنی «چه چیزی + کجا»، و اینکه سختیِ اصلی از تعدادِ نامعلومِ خروجی‌ها می‌آید. جعبه فقط چهار عدد است و مکان‌یابیِ تک‌شیء یک مسئلهٔ رگرسیون؛ پنجرهٔ لغزانِ روی هرمِ تصویر تشخیصِ چندشیء را ممکن اما کند می‌کند؛ **IoU** هم‌پوشانی را می‌سنجد و **NMS** جعبه‌های تکراری را جمع می‌کند؛ آشکارسازهای مدرن از پیشنهادِ ناحیه تا تک‌شاتِ YOLO همان جاروب را کارآمد کرده‌اند و هنوز به IoU و NMS تکیه دارند؛ و آشکارساز را باید با precision/recall و mAP سنجید، نه با دقتِ ساده. مهم‌تر از همه، اکنون می‌توانیم نوارِ ارقام را «پیدا کنیم، بعد بخوانیم» و خطِ‌لولهٔ DocScan را برای فینالِ درس — یادگیریِ انتقالی و پروژهٔ نهایی — آماده کنیم.